Adversarial Suffix Optimization for Jailbreaking Large Language Models: A Greedy Coordinate Simulated Annealing Approach

Ceren Duru Çınar

March 24, 2026

1 Introduction

Large Language Models (LLMs) have rapidly advanced in recent years, enabling breakthroughs in natural language generation, translation, and reasoning. However, their widespread deployment raises pressing security concerns. Despite alignment training and safety measurements, LLMs remain vulnerable to *jailbreaks*, where carefully designed adversarial prompts circumvent safeguards and elicit restricted or unsafe responses. Such adversarial manipulations represent a growing risk, as documented by Carlini and Wagner [1], Zou et al. [2], and subsequent works.

Adversarial suffix attacks in particular demonstrate that short token sequences appended to malicious prompts can reliably induce harmful outputs. These attacks are concerning because they can be performed in black-box settings, requiring only query access to the model. As Hayase et al. [3] recently demonstrated, query-based optimization methods can achieve high success rates without gradient information, even against production models.

This study builds upon their method, *Query-Based Adversarial Prompt Generation*, and proposes a variant algorithm incorporating simulated annealing. The new method, termed *Greedy Coordinate Simulated Annealing* (GC-SA), is compared empirically against the original Greedy Coordinate Query (GCQ) method.

2 Background

Adversarial examples were first studied in the vision domain, where small perturbations to images were introduced to induce misclassification [4, 5]. In natural language processing, the discrete nature of text makes direct gradient-based optimization more difficult. Early work explored character- or word-level substitutions [6], while later approaches developed embedding-space perturbations [7].

In the context of LLMs, prompt-based adversarial attacks have emerged as a new category. Zou et al. [2] introduced universal and transferable adversarial suffixes capable of jailbreaking multiple models. More recently, Hayase et al. [3] presented GCQ, a query-efficient algorithm that leverages a buffer-based search to find adversarial suffixes without access to gradients.

3 Summary of GCQ

The Greedy Coordinate Query (GCQ) algorithm [3] extends the earlier Greedy Coordinate Gradient (GCG) method [2] by removing the need for gradient information. GCQ maintains a buffer of candidate adversarial suffixes and iteratively refines them using query-based evaluations. At each step:

- A suffix is sampled from the buffer and locally perturbed at one position.
- Candidate replacements are scored using a proxy loss, with the best candidates promoted.
- The buffer is updated to retain only the lowest-loss examples.

This design yields a surrogate-free, black-box attack that achieves high success rates on both open-source and API-hosted models. Hayase et al. report success rates exceeding 79% on GPT-3.5 with modest query budgets.

4 Proposed Method: GC–SA

This section details a simulated annealing variant of coordinate-wise adversarial suffix search. The method preserves the standard black-box setting (no gradient access, and forward passes only), but augments greedy coordinate updates with a temperature-controlled acceptance rule to escape local minima.

4.1 Algorithm Overview

Let P denote the base prompt, $T = [t_1, \ldots, t_n]$ the target token sequence, and $S = [s_1, \ldots, s_k]$ the learnable suffix. The objective minimizes the target-only negative log-likelihood (NLL):

$$\mathcal{L}(S\,; P, T) \;=\; -\frac{1}{n}\sum_{i=1}^{n} \log p\big(t_i \mid (P{+}S)_{<\,|P|+|S|+i}\big)\,.$$

Each iteration proceeds in three phases:

1. **Coordinate selection:** For each position $j \in \{1, \ldots, k\}$, sample one (or more) candidate replacements from a proposal distribution restricted to an allowed token set $\mathcal{A}$ (Section 4.6). Score the best trial at each j to identify an index $j^\star$ that yields the largest decrease in $\mathcal{L}$.
2. **Local refinement:** At $j^\star$, evaluate b additional candidates drawn from $\mathcal{A}$. Let $\tilde{s}$ be the best local token and $\widetilde{\mathcal{L}}$ its loss.
3. **Annealed acceptance:** If $\widetilde{\mathcal{L}} < \mathcal{L}(S)$, accept deterministically. Otherwise accept with probability $\exp(-\Delta/T_{\text{eff}})$, where $\Delta = \widetilde{\mathcal{L}} - \mathcal{L}(S)$ and $T_{\text{eff}} = T_0 \cdot \gamma^t$ follows an exponential cooling schedule with iteration index t.

4.2 Initialization and Schedules

Initialization: S is initialized by repeating (and truncating) the tokenization of T until length k is reached. This produces an on-manifold seed that empirically accelerates early loss reduction [3].

Cooling: The effective temperature is $T_{\text{eff}} = T_0 \cdot \gamma^t$ with $T_0 > 0$ and $\gamma \in (0, 1)$. Larger T_0 or

smaller γ encourages exploration early and reduces it over time.

Stopping: The search stops after T iterations or when no acceptance occurs for m consecutive iterations.

4.3 Candidate Generation and Scoring

Candidates at position j are drawn from a constrained set $\mathcal{A}$ (Section 4.6). For a trial token $u \in \mathcal{A}$, let $S^{(j \leftarrow u)}$ denote S with s_j replaced by u. The score is the target-only NLL $\mathcal{L}(S^{(j \leftarrow u)}; P, T)$ computed by a single forward pass on the concatenation $P + S^{(j \leftarrow u)} + T$ while only accumulating loss on T.

4.4 Pseudocode

Algorithm 1 Greedy Coordinate Simulated Annealing (GC–SA)

Require: M decoder-only language model, P prompt sequence, $T = [t_1, \ldots, t_n]$ target sequence, k suffix length, $T_{\max}$ maximum number of iterations, b local refinement width, T_0 initial temperature, $\gamma \in (0, 1)$ temperature decay factor, $\mathcal{A} \subseteq \mathcal{V}$ allowed candidate token set.

Initialize suffix $S = [s_1, \ldots, s_k]$ by repeating the tokenization of T and truncating to length k
Compute initial loss $L \leftarrow \mathcal{L}(S; P, T)$
for $t = 1$ to $T_{\max}$ **do**
  $T_{\text{eff}} \leftarrow T_0 \cdot \gamma^t$
  Set $j^\star \leftarrow -1$, $L^\star \leftarrow +\infty$, $u^\star \leftarrow \varnothing$
  for $j = 1$ to k **do**
    Sample a candidate token $u \in \mathcal{A}$, with $u \neq s_j$
    Form trial suffix $S^{(j)}$ with $s_j \leftarrow u$
    Compute loss $L_j \leftarrow \mathcal{L}(S^{(j)}; P, T)$
    if $L_j < L^\star$ **then**
      $L^\star \leftarrow L_j$, $j^\star \leftarrow j$, $u^\star \leftarrow u$
  Initialize $\widetilde{L} \leftarrow L^\star$, $\tilde{u} \leftarrow u^\star$
  for $r = 1$ to b **do**
    Sample $v \in \mathcal{A}$, with $v \neq s_{j^\star}$
    Form trial suffix $\widehat{S}$ with $s_{j^\star} \leftarrow v$
    Compute loss $L_v \leftarrow \mathcal{L}(\widehat{S}; P, T)$
    if $L_v < \widetilde{L}$ **then**
      $\widetilde{L} \leftarrow L_v$, $\tilde{u} \leftarrow v$
  $\Delta \leftarrow \widetilde{L} - L$
  if $\Delta < 0$ **or** $\text{Uniform}(0, 1) < \exp(-\Delta / T_{\text{eff}})$ **then**
    Accept update: $s_{j^\star} \leftarrow \tilde{u}$, $L \leftarrow \widetilde{L}$
  else
    Reject update: retain S and L
return optimized suffix S

4.5 Generation Configuration

After optimization, generations are elicited with a two-phase configuration that balances diversity and determinism:

- **Decoder settings.** *max_new_tokens* $\approx$ 500; *temperature* 0.7; *top_p* 0.9; *do_sample* `True`; *eos_token_id* and *pad_token_id* set according to the model configuration.
- **Hybrid decoding.** A short stochastic preamble (e.g., 5 sampling steps) is followed by greedy continuation until *eos* or token budget is reached. This pattern reduces mode collapse while keeping completions comparable across trials.

All reported results are computed on the raw decoded text produced under this configuration.

4.6 Token Filtering and Constraints

To regularize the search space and avoid degenerate proposals, the candidate token set $\mathcal{A} \subset \mathcal{V}$ is restricted using lightweight filters:

1. **Printability.** Disallow non-printable or control characters; retain tokens whose decoded form lies within standard ASCII printable ranges.
2. **Lexicality.** Retain tokens decoding to alphabetic strings (`[A{Z a{z]+`). An optional English lexicon check may be applied to further down-select plausible morphemes.
3. **Sanitization.** Exclude empty/whitespace-only decodes and collapse repeated spaces. A deduplication pass prevents proposing the current token at j.

These filters produce a stable $\mathcal{A}$ that empirically improves convergence and reduces wasted evaluations, without relying on gradients or external scorers.

4.7 Complexity and Practical Settings

Each outer iteration performs $(k{+}b)$ loss evaluations. With $T_{\max}$ iterations, the total number of forward passes is $O(T_{\max}(k{+}b))$. In experiments, typical settings are $T_{\max}{=}400$, $k{=}16$, $b{=}20$, which fit comfortably within a single-GPU budget for decoder-only LLMs.

5 Experimental Setup

5.1 Dataset

Evaluation was conducted on a dataset of **131 adversarial prompts** paired with predefined target strings, from harmful behaviors dataset [2].

5.2 Evaluation Metric

Outputs were classified using a refusal-prefix heuristic. Any output containing refusal markers such as "I'm sorry" or "As an AI" was labeled as non-jailbroken; otherwise, it was marked as jailbroken.

5.3 Implementation

The GC-SA algorithm was run with 400 iterations, suffix length of 16, and beam width of 20. For comparison, the same dataset was tested using the GCQ method [3].

6 Results

Table 1 summarizes the results. Both GC-SA and GCQ achieved a jailbreak success rate of approximately 78% across the 131 prompts. As the official GCQ implementation was not publicly released, the method was re-implemented for the purposes of this evaluation. It should be noted that jailbreak success rates are not static and may vary as underlying models are updated and trained with stronger alignment techniques.

Method	Prompts Tested	Jailbreak Success Rate
GC-SA (proposed)	131	∼78%
GCQ [3]	131	∼78%

Table 1: Comparison of GC-SA and GCQ on 131 prompts using the refusal-prefix heuristic.

Figure 1 illustrates a representative loss trajectory, where rapid improvements occur in early iterations before plateauing.

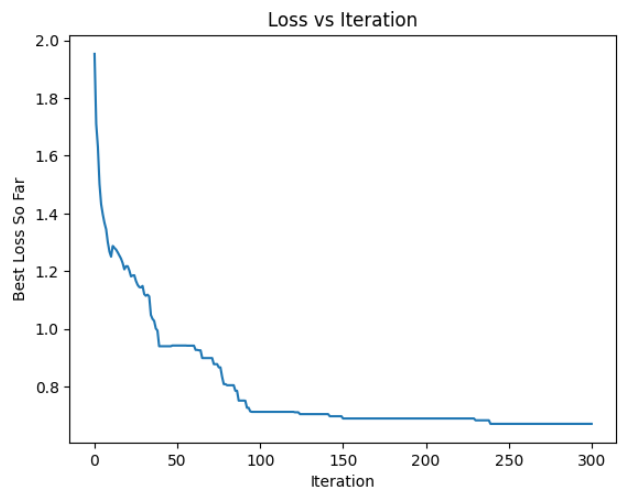

Figure 1: Loss trajectory for one example prompt under GC-SA.

7 Exploration Analysis

Beyond aggregate success rates, it is informative to analyze how the search distributes attention across suffix positions during optimization. Figure 2 shows a heatmap of the per-iteration selection rate for each suffix position ($j = 0, \ldots, 15$), alongside a bar plot of the average number of times each position was selected per example across the dataset.

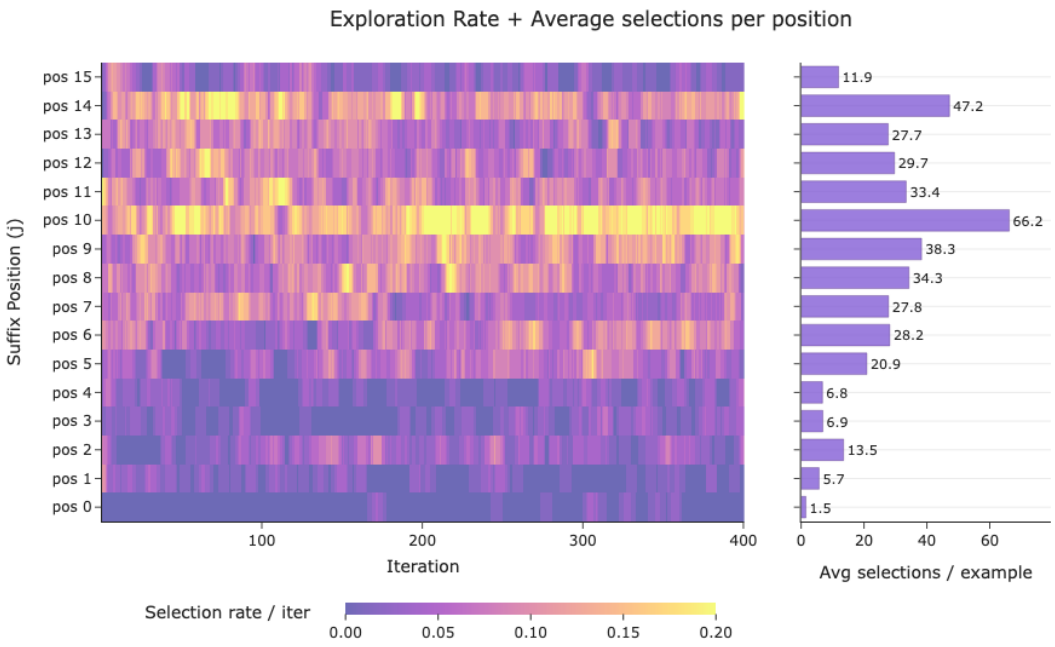

Figure 2: Exploration dynamics during GC-SA optimization. Left: selection rate of suffix positions across 400 iterations. Right: average number of selections per position.

The heatmap indicates that later suffix positions (particularly positions 8–14) are sampled more frequently than earlier positions. The bar plot confirms this imbalance, with position 10 being selected on average 66.2 times per example, while initial positions are rarely updated.

This pattern suggests that the optimization process concentrates its search on the central and later parts of the suffix. Early positions (e.g., positions 0–2) are modified far less often, likely because tokens closer to the prompt boundary exert less influence on the likelihood of the target continuation. Conversely, tokens closer to the end of the suffix strongly condition the immediate transition into the target sequence, and thus offer greater leverage for reducing loss.

The results highlight two points:

1. The optimization dynamics are highly position-dependent, with middle-to-late suffix positions driving most of the improvement.

2. Future algorithmic refinements could prioritize exploration of these critical positions, or adaptively reweight updates based on observed influence, potentially reducing query complexity.

8 Conclusion

This report examined adversarial suffix optimization for jailbreaking large language models under black-box conditions. Building on the Query-Based Adversarial Prompt Generation (GCQ) method [3], a variant incorporating simulated annealing was introduced. The proposed GC-SA algorithm modifies the coordinate-wise update step by accepting occasional suboptimal replacements according to a decaying temperature schedule, thereby encouraging exploration of the token search space.

Evaluation was conducted on a dataset of 131 adversarial prompts, using a refusal-prefix heuristic to classify jailbreak success. GC-SA achieved a jailbreak success rate of approximately 78%, which is essentially identical to the performance of the original GCQ algorithm on the same dataset. These results indicate that GCQ is already close to optimal for this setting, and that simulated annealing provides a comparably effective alternative update strategy.

The findings contribute two insights. First, the robustness of suffix-based jailbreaks across algorithmic variations underscores the vulnerability of current alignment techniques to prompt-based adversarial search. Second, the comparable performance of GC-SA and GCQ suggests that the design space of black-box search heuristics is rich, and small modifications may not substantially alter success rates without additional structural innovations.

Future work may extend this analysis by testing GC-SA and GCQ across larger and more diverse prompt corpora, incorporating semantic evaluation metrics beyond refusal-prefix heuristics, and exploring hybrid search strategies that combine annealing with adaptive candidate generation. Such investigations would clarify whether simulated annealing offers unique advantages in more complex or resource-constrained adversarial settings.

References

[1] Carlini, N. and Wagner, D. Towards evaluating the robustness of neural networks. In *2017 IEEE Symposium on Security and Privacy*, 2017.

[2] Zou, A., Wang, Z., Carlini, N., Nasr, M., Kolter, J. Z., and Fredrikson, M. Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043*, 2023.

[3] Hayase, J., Borevkovic, E., Carlini, N., Tramèr, F., and Nasr, M. Query-Based Adversarial Prompt Generation. *arXiv preprint arXiv:2402.12329*, 2024. doi: 10.48550/arXiv.2402.12329.

[4] Szegedy, C., Zaremba, W., Sutskever, I., Bruna, J., Erhan, D., Goodfellow, I., and Fergus, R. Intriguing properties of neural networks. In *International Conference on Learning Representations*, 2014.

[5] Goodfellow, I., Shlens, J., and Szegedy, C. Explaining and harnessing adversarial examples. In *International Conference on Learning Representations*, 2015.

[6] Li, J., Ji, S., Du, T., Li, B., and Wang, T. TextBugger: Generating adversarial text against real-world applications. *arXiv preprint arXiv:1812.05271*, 2018.

[7] Guo, C., Sablayrolles, A., Jégou, H., and Kiela, D. Gradient-based adversarial attacks against text transformers. *arXiv preprint arXiv:2104.13733*, 2021.